

Assignment – 19.2

Name: Saisharan

Ht.no: 2303A51434

bt.no: 21

Task 1 – Python to Java Conversion

Task:

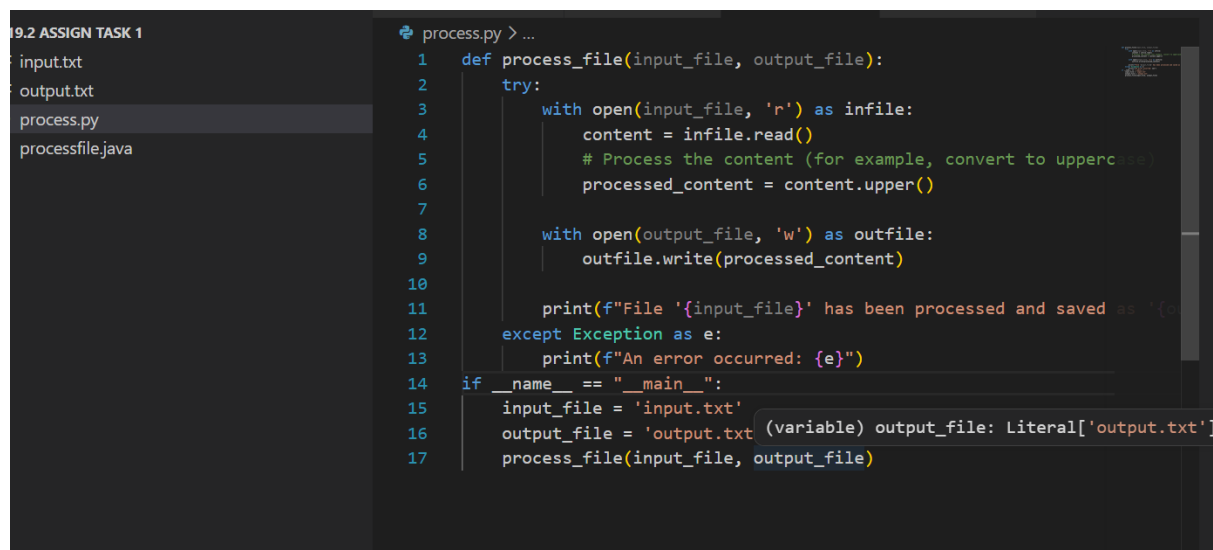
Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

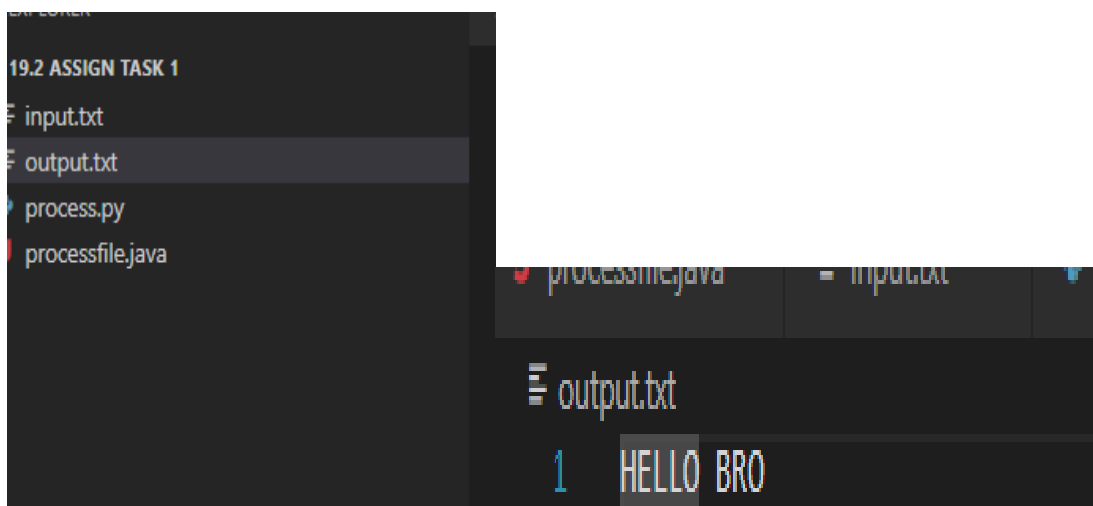
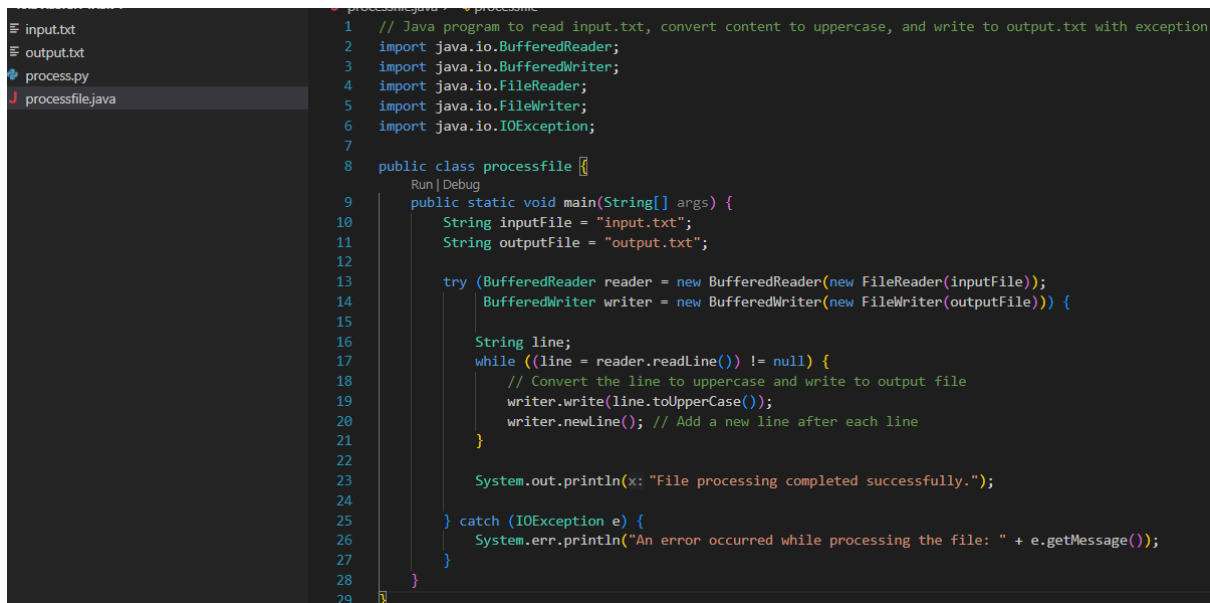
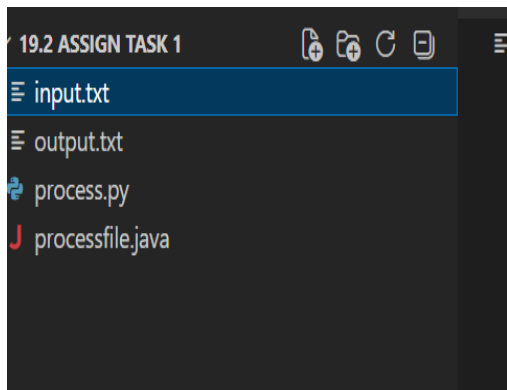
Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.



```
19.2 ASSIGN TASK 1
input.txt
output.txt
process.py
processfile.java

process.py > ...
1 def process_file(input_file, output_file):
2     try:
3         with open(input_file, 'r') as infile:
4             content = infile.read()
5             # Process the content (for example, convert to uppercase)
6             processed_content = content.upper()
7
8         with open(output_file, 'w') as outfile:
9             outfile.write(processed_content)
10
11         print(f"File '{input_file}' has been processed and saved as '{output_file}'")
12     except Exception as e:
13         print(f"An error occurred: {e}")
14 if __name__ == "__main__":
15     input_file = 'input.txt'
16     output_file = 'output.txt'
17     process_file(input_file, output_file)
```



Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Code :

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4 void bubbleSort(vector<int>& arr) {
5     int n = arr.size();
6     for (int i = 0; i < n - 1; i++) {
7         for (int j = 0; j < n - i - 1; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 swap(arr[j], arr[j + 1]);
10            }
11        }
12    }
13 }
14 void printArray(const vector<int>& arr) {
15     for (int num : arr) {
16         cout << num << " ";
17     }
18     cout << endl;
19 }
20 int main() {
21     // Test case 1: Basic test case
22     vector<int> arr1 = {64, 34, 25, 12, 22, 11, 90};
23     cout << "Original array: ";
24     printArray(arr1);
25     bubbleSort(arr1);
26     cout << "Sorted array: ";
27     printArray(arr1);
28
29     // Test case 2: Already sorted array
30     vector<int> arr2 = {1, 2, 3, 4, 5};
31     cout << "Original array: ";
32     printArray(arr2);
33     bubbleSort(arr2);
34     cout << "Sorted array: ";
35     printArray(arr2);
36
37     // Test case 3: Reverse sorted array
38     vector<int> arr3 = {5, 4, 3, 2, 1};
```

```
File Edit Selection View ... 19.2
RUN AND DEBUG ...
task 2\bubble.cpp • 2 problems in this file
2 n = len(arr)
3 # Traverse through all elements in the array
4 for i in range(n):
5     # Last i elements are already in place, no need to check them
6     for j in range(0, n-i-1):
7         # Swap if the element found is greater than the next element
8         if arr[j] > arr[j+1]:
9             arr[j], arr[j+1] = arr[j+1], arr[j]
10 # Example usage
11 if __name__ == "__main__":
12     arr = [64, 34, 25, 12, 22, 11, 90]
13     print("Original array:", arr)
14     bubble_sort(arr)
15     print("Sorted array:", arr)
16
```

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

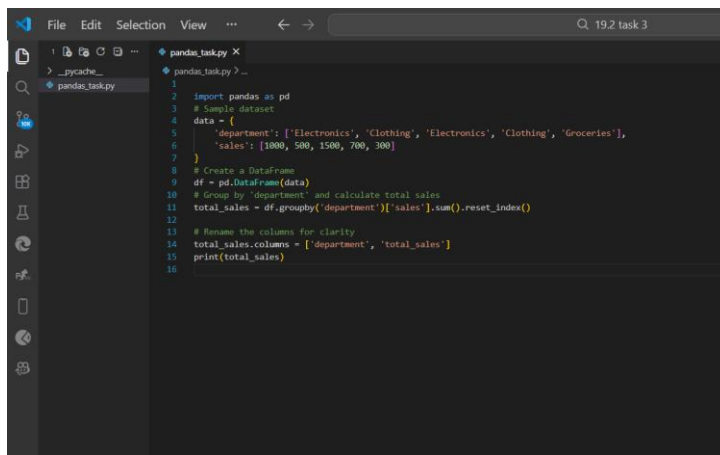
- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).

- Prompt AI to generate equivalent Pandas code using `groupby()` and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Code:



```

1
2 import pandas as pd
3 # Sample dataset
4 data = {
5     'department': ['Electronics', 'Clothing', 'Electronics', 'Clothing', 'Groceries'],
6     'sales': [1800, 500, 1500, 700, 300]
7 }
8 # Create a DataFrame
9 df = pd.DataFrame(data)
10 # Group by 'department' and calculate total sales
11 total_sales = df.groupby('department')['sales'].sum().reset_index()
12
13 # Rename the columns for clarity
14 total_sales.columns = ['department', 'total_sales']
15 print(total_sales)
16

```

Output:

	department	total_sales
0	Clothing	1200
1	Electronics	2500
2	Groceries	300

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Code:

```
python.py > ...
1  # Calculate total and average marks
2
3  students = [
4      {"name": "A", "marks": [80, 90, 85]},
5      {"name": "B", "marks": [70, 60, 75]},
6      {"name": "C", "marks": [88, 92, 95]}
7  ]
8
9  for student in students:
10     total = sum(student["marks"])
11     avg = total / len(student["marks"])
12
13     print(f"{student['name']} -> Total: {total}, Average: {avg:.2f}")
```

```
cppvenkapp1 @ ming
1
2  #include <iostream>
3  using namespace std;
4  void displayMenu() {
5      cout << "Simple Calculator" << endl;
6      cout << "1. Addition" << endl;
7      cout << "2. Subtraction" << endl;
8      cout << "3. Multiplication" << endl;
9      cout << "4. Division" << endl;
10     cout << "5. Exit" << endl;
11 }
12 int main() {
13     int choice;
14     double num1, num2;
15
16     do {
17         displayMenu();
18         cout << "Enter your choice: ";
19         cin >> choice;
20
21         if (choice >= 1 && choice <= 4) {
22             cout << "Enter two numbers: ";
23             cin >> num1 >> num2;
24         }
25
26         switch (choice) {
27             case 1:
28                 cout << "Result: " << num1 + num2 << endl;
29                 break;
30             case 2:
31                 cout << "Result: " << num1 - num2 << endl;
32                 break;
33             case 3:
34                 cout << "Result: " << num1 * num2 << endl;
35                 break;
36             case 4:
37                 if (num2 != 0) {
38                     cout << "Result: " << num1 / num2 << endl;
39                 } else {
40                     cout << "Error: Division by zero is not allowed." << endl;
41                 }
42                 break;
43             case 5:
44                 cout << "Exiting the calculator. Goodbye!" << endl;
45                 break;
46             default:
47                 cout << "Invalid choice. Please try again." << endl;
48         }
49     } while (choice >= 1 && choice <= 5);
50 }
```

Task -5: (Basic Function Translation (Python to C++))

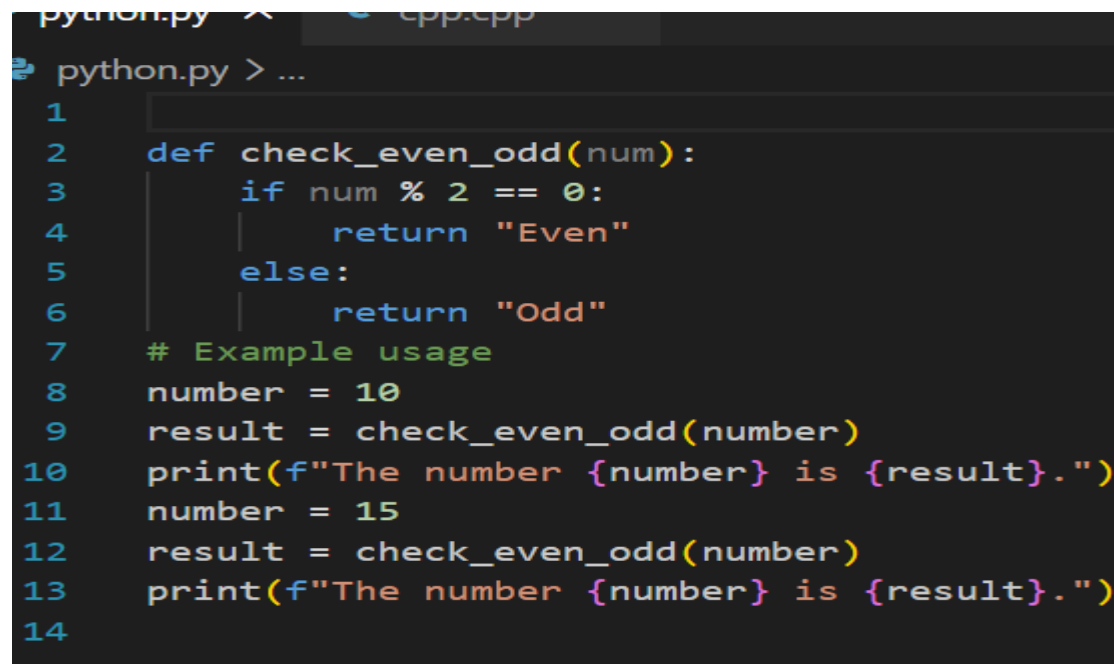
Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

A screenshot of a code editor with a dark background. The editor shows a Python file named 'python.py' with the following code:

```
1
2 def check_even_odd(num):
3     if num % 2 == 0:
4         return "Even"
5     else:
6         return "Odd"
7 # Example usage
8 number = 10
9 result = check_even_odd(number)
10 print(f"The number {number} is {result}.")
11 number = 15
12 result = check_even_odd(number)
13 print(f"The number {number} is {result}.")
14
```

```
cpp.cpp 2
cpp.cpp > ...
1  #include <iostream>
2  using namespace std;
3  bool isEven(int number) {
4      return number % 2 == 0;
5  }
6  int main() {
7      int num;
8      cout << "Enter a number: ";
9      cin >> num;
10
11     if (isEven(num)) {
12         cout << num << " is even." << endl;
13     } else {
14         cout << num << " is odd." << endl;
15     }
16
17     return 0;
18 }
19
```

Output:

```
The number 10 is Even.
The number 15 is Odd.
```